PURPLE TEAM HOME LAB · TECHNICAL REPORT

Lab v3 — Network Detection and Host EDR Layers

A full Active Directory kill chain run against five telemetry sources, then measured: what each layer caught, what it missed, and which of the shipped detections could not have fired at all.

Domain MARVEL.LOCAL · 7 hosts · VMware NAT 192.168.182.0/24

Attack run 20 October 2026 · **Rule verification** 24 October 2026

Author Stevan Mrkšić, self-taught, Belgrade · linkedin.com/in/stevan92mrksic

Training TCM Security: SOC 101, SOC 201, Detection Engineering for Beginners, Practical Windows Forensics, Practical Ethical Hacking. In progress: Practical Malware Analysis and Triage.

Repository github.com/slim92code/purple-team-lab

Contents

1. Summary — what v3 adds, and the four findings that matter

2. Infrastructure — hosts, telemetry sources, topology

3. Hardening baseline as built — the firewall mesh and where it leaked

4. Attack chain — thirteen steps, Defender enabled throughout

5. Detection coverage — fifteen detections across five sources

6. Findings — eight, each with the measurement behind it

7. Detection engineering — rules shipped, tuned and verified

8. Alert triage — seven case files

9. Known gaps carried into v4

Appendix A — rule verification log, 24 October 2026

Appendix B — measurement data

1. Summary

v3 adds two detection layers to the v2 Sysmon and Splunk baseline: network detection with Zeek and Suricata, and host EDR with Wazuh. Five telemetry sources feed one SIEM. The same kill chain was re-run and each phase checked against every source.

The interesting part is not that the attack was detected. It is what the measurement afterwards showed about the detections themselves. Four of the rules in the stack could not have alerted on this attack, each for a different reason, and all four looked correct until they were tested. A fifth alerted correctly and still told the analyst nothing.

Rule	Why it could not alert on this attack
Custom Wazuh LSASS rule	bound to a rule group that does not exist, so it was loaded and discarded
Built-in Wazuh `92900`	filters `grantedAccess` to the mask Defender uses, not the one a dump uses
RC4 Kerberoasting rule	this domain issues AES only, so the rule has never had an input
C2 beacon (CIM `tstats`)	queries a data model that holds zero events on this forwarder configuration
and built-in `60228`	*alerted at level 4, the same score as a OneDrive updater*

What changed from v2

Area	v2	v3
Telemetry sources	Windows Security log, Sysmon	plus Zeek, Suricata, Wazuh
Kerberos encryption	RC4 (0x17) roast	AES-256 (0x12) roast
Initial access lands on	a Domain Admin, so no escalation path	`pparker`, a standard user, who then roasts a DA SPN
Defender exclusions	`C:\Temp` excluded for reproducible delivery	none; `Get-MpPreference` shows no exclusion paths
Splunk licence	Free, so searches run on demand	Enterprise, so alerts fire on schedule
Deliverable	attack narrative and detection cards	seven alert-triage case files

The four findings that matter

> **A canonical detection that never saw its input.** The RC4 Kerberoasting rule keys on `Ticket_Encryption_Type = 0x17`. Counted over 30 days in this lab: 1,649 service tickets, every one of them `0x12`, zero RC4. The rule is scheduled, it sits in the coverage table, and it has never had an input. Not because the domain refuses RC4 — the Pass-the-Ticket step got an RC4 TGT for the same account — but because the roast requested no particular encryption type and the KDC issued the strongest one available. It is one of three rules in this lab that reported coverage and could not deliver it; the other two are in 6.4 and 6.8.

> **An EDR that stayed silent on a credential dump, for four separate reasons.** The other endpoint cannot produce the event at all; the EDR never receives Sysmon from two of three agents; the built-in

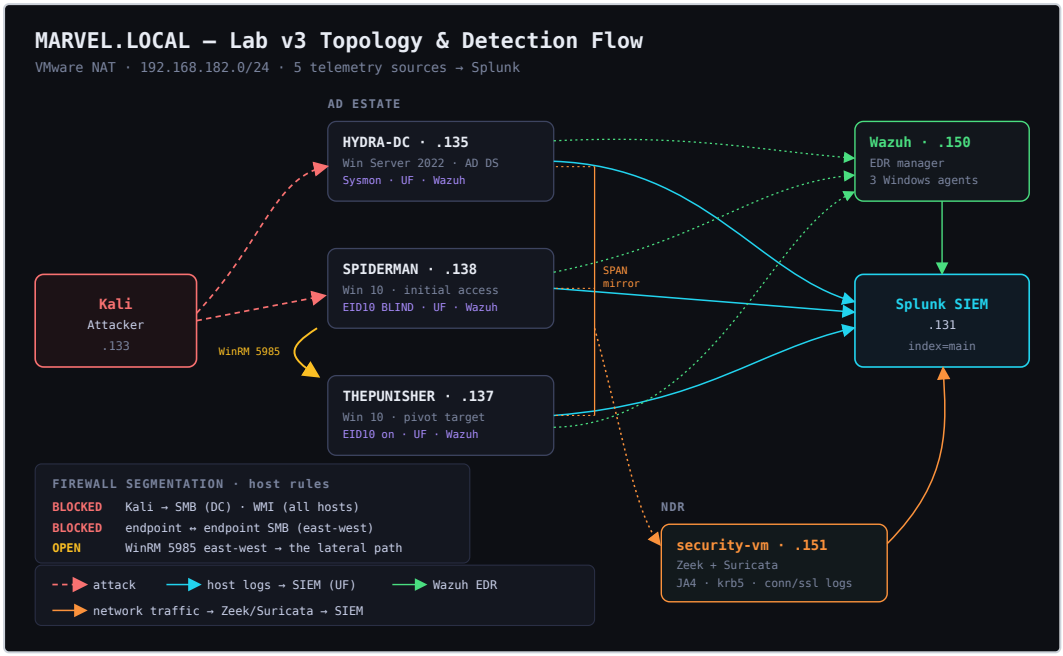
> LSASS rule filters to the access mask Defender uses rather than the one a dump uses; and the custom rule written to fix that bound to a rule group which does not exist, with the engine printing the warning on every start.

> **An alert that worked and still told the analyst nothing.** SYSTEM-level persistence with a PowerShell download cradle arrived at level 4, the same score the same rule gave a OneDrive updater. Scoring task creation without scoring task content discards the only part that separates the two.

> **A showcase detection querying an empty data model.** The behavioural C2 rule runs against `Network_Traffic.All_Traffic`, which holds zero events here. 444 Sysmon EID 3 events reach the index in seven days carrying no CIM fields, because the forwarder ships every Windows channel as a bare `WinEventLog` sourcetype and the Sysmon add-on keys its mappings on the per-channel sourcetype.

Each of these was found by measuring rather than by reading the rule. Each fix is verified by triggering the condition rather than by the absence of alerts. Section 7 and Appendix A carry the verification log.

2. Infrastructure



Attack paths in red, the WinRM pivot in amber, Wazuh agent channels in green, and the SPAN mirror feeding Zeek and Suricata. The firewall panel shows what the segmentation mesh blocks and what it leaves open.

Host	Address	Role
HYDRA-DC	`.135`	Windows Server 2022, AD DS, `MARVEL.LOCAL`
SPIDERMAN	`.138`	Windows 10 endpoint
THEPUNISHER	`.137`	Windows 10 endpoint
Splunk	`.131`	Splunk 10.2, Enterprise licence
Wazuh	`.150`	Wazuh 4.12 manager, 3 Windows agents
security-vm	`.151`	Zeek 9.0.0 and Suricata 6.0.4
Kali	`.133`	Attacker

Telemetry sources and what each one is good for

Source	Covers	Blind to
Windows Security log	authentication, process creation, task creation	an authentication attempt against an account that does not exist
Sysmon	process, network and handle-level detail on the host	whatever the per-host config does not include
Wazuh	host EDR, rule scoring on arriving events	channels the agent is not configured to read
Zeek	connection and protocol metadata, both outcomes of an exchange	anything that does not cross the mirrored link
Suricata	application-layer anomalies and signature matches	encrypted payload content

> **Agent count is not telemetry coverage.** Wazuh reports three healthy Windows agents. Across every archived alert file, only one of them has ever sent a Sysmon event. Section 6.4 carries the numbers.

3. Hardening baseline as built

The Windows estate is not a default install. A host-firewall segmentation mesh and SMB hardening were applied before the run, so the chain had to work through them.

Host	Blocks inbound, by explicit rule
HYDRA-DC	Kali to SMB, Kali to WMI
THEPUNISHER	SPIDERMAN to SMB, Kali to WMI
SPIDERMAN	THEPUNISHER to SMB, Kali to WMI

Also applied: SMBv1 removed on all three hosts, SMB signing required on the DC, account lockout at five attempts, and RemoteRegistry plus several discovery and legacy services disabled on the endpoints.

Where the protection actually comes from

SMB from the attacker to the two endpoints is blocked as well, but by the Windows inbound default rather than by a rule: their firewall profiles are `NotConfigured`, which means block. The DC's inbound default is `Allow`, so there the explicit rule is the only thing stopping it. One host is protected by a rule that someone wrote, two by a default nobody set.

Where it held and where it leaked

The mesh moved the attack off SMB and WMI. WinRM on 5985 was left open east-west, and that is the path the lateral movement took. Prevention narrowed the attack surface to a single predictable lane, and the detection layer was watching that lane. Narrowing is worth doing even when it does not block, provided something watches what is left open.

Residual gaps, deliberate, and the v4 before-state

Gap	Consequence
WinRM open east-west	the lateral-movement path; kept by design as the monitored pivot
SMB signing only on the DC	endpoints accept unsigned SMB
DC inbound default is `Allow`	only an explicit rule blocks the attacker there
No LSA Protection (`RunAsPPL`)	the LSASS dump succeeds, which is what section 5 detects
WDigest and LLMNR not hardened	credential-exposure and name-poisoning surface

4. Attack chain

The red-team side of the run. Each step produced the telemetry evaluated in section 5. Defender was enabled throughout, with no exclusion paths, which step 13 shows.

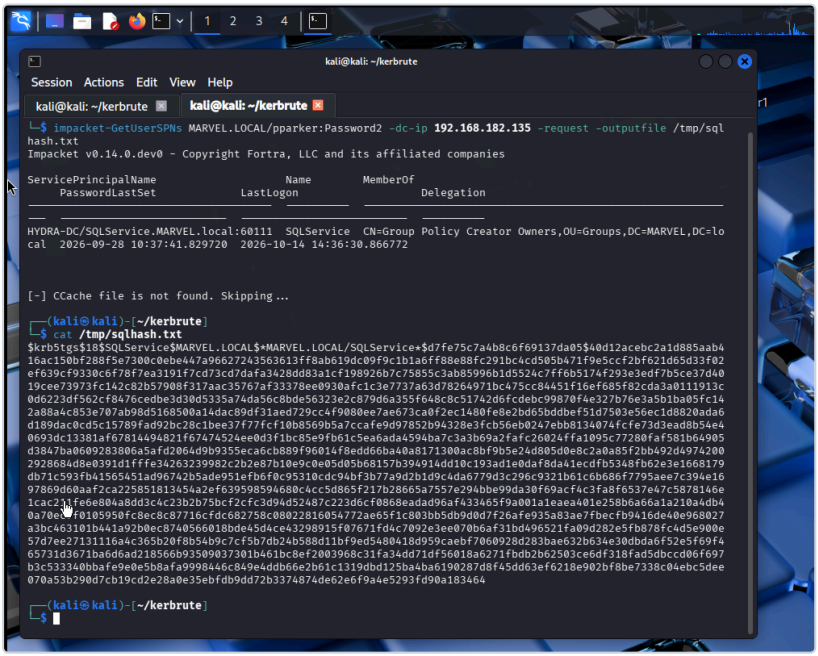
Lab credentials and a deliberately weak password are shown as captured. This is an isolated `MARVEL.LOCAL` lab.

Step 1 — Password spray (T1110.003)



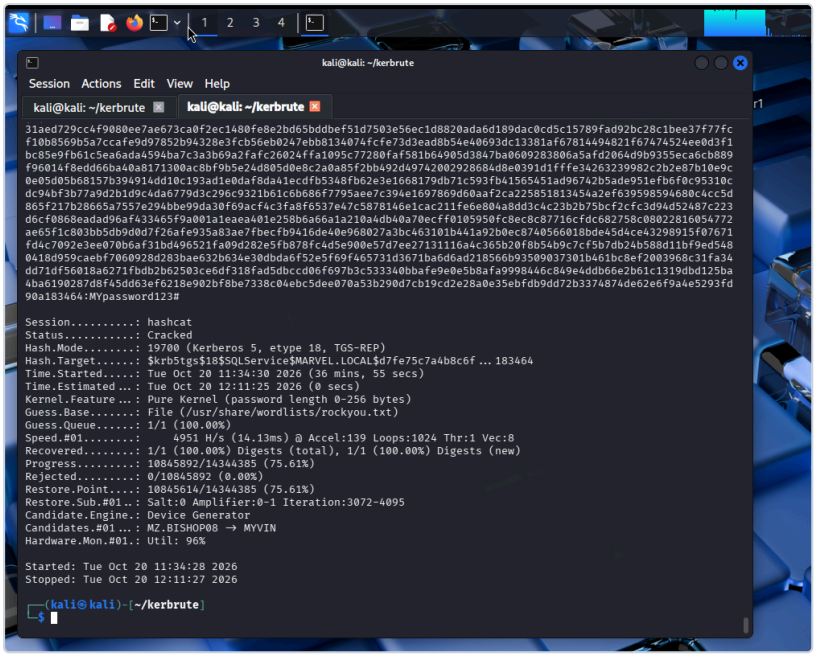
`kerbrute passwordspray` against the DC with one password across a user list. Seven logins tested, `pparker` valid. One attempt per account, which stays under the lockout threshold of five.

Step 2 — Kerberoasting, SPN ticket request (T1558.003)



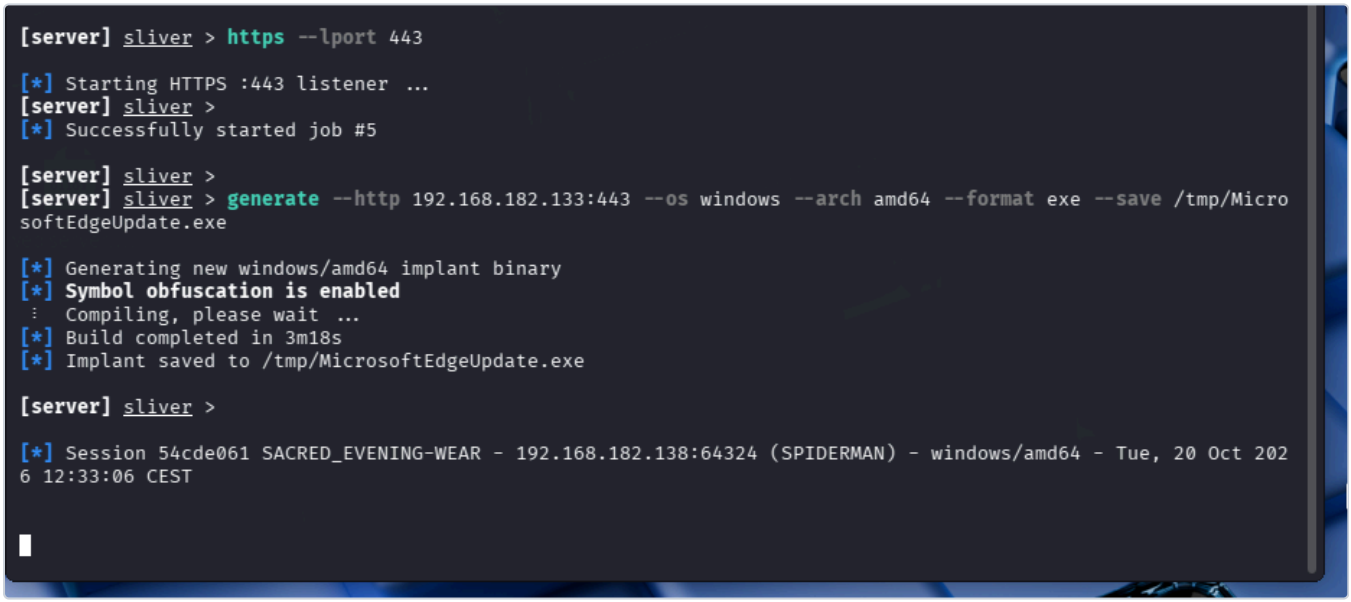
`impacket-GetUserSPNs` with the `pparker` credential requests the `SQLService` ticket and writes the `krb5tgs` hash.

Step 3 — Kerberoasting, offline crack (T1558.003)



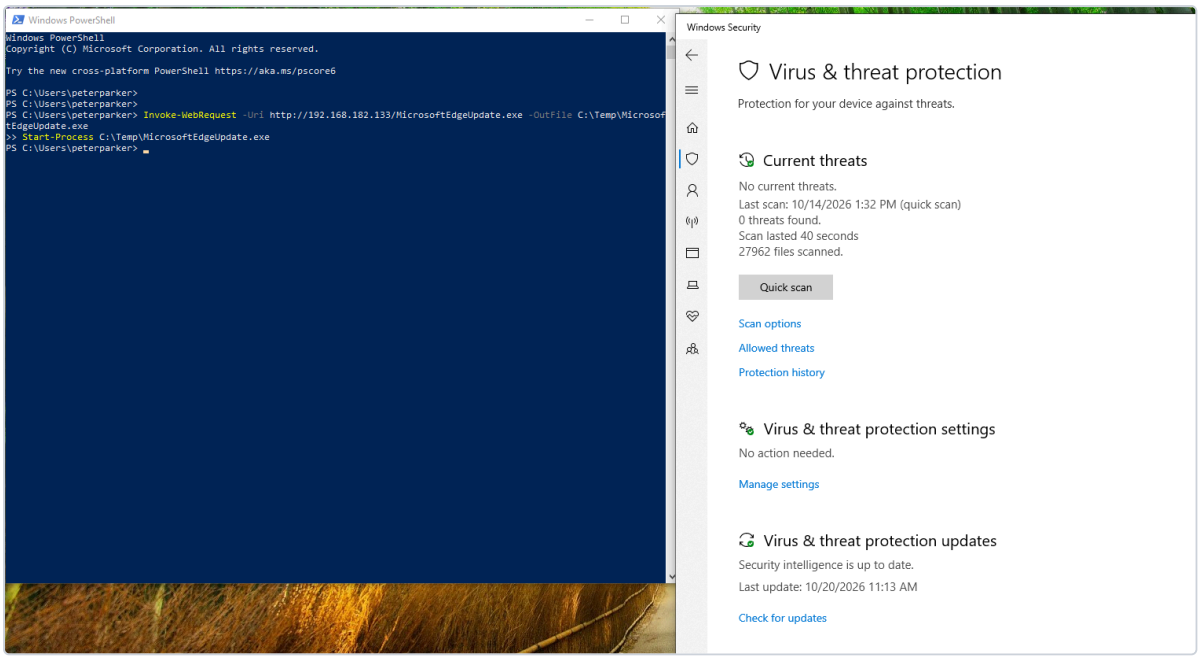
hashcat mode 19700, Kerberos 5 AES-256 TGS-REP, cracks the `SQLService` hash from a wordlist. The mode number is the point: this is an AES roast, not RC4.

Step 4 — C2 listener and implant (T1071.001)



Sliver HTTPS listener started and a Windows implant generated. The session from SPIDERMAN checks in.

Step 5 — Payload delivery (T1105)



The implant is downloaded to the endpoint and run. The Defender pane shows no detection, so what catches this is the detection layer rather than prevention.

Step 6 — Pass-the-Ticket (T1550.003)



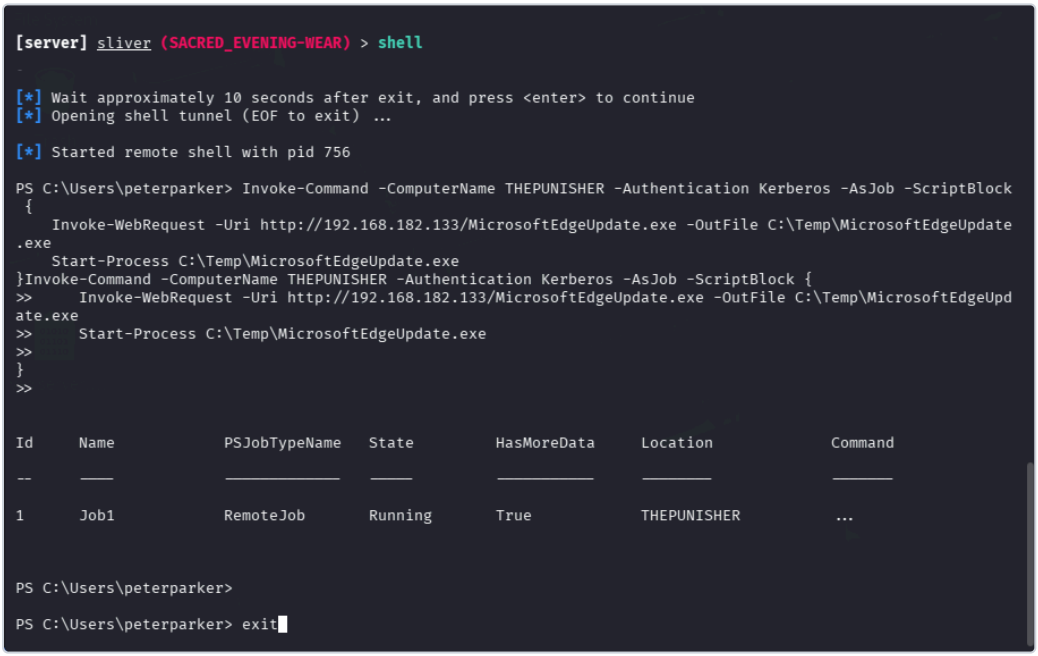
Rubeus run in memory through Sliver `execute-assembly`, requesting a TGT for `SQLService` and injecting it with `/ptt`.

Step 7 — Pass-the-Ticket, ticket imported (T1550.003)



The imported ticket with its flags and lifetime. The credential is now usable without the account password.

Step 8 — Lateral movement over WinRM (T1021.006)



`Invoke-Command -ComputerName THEPUNISHER -Authentication Kerberos` runs a remote job using the injected ticket. This is the port-5985 traffic in detection 05.

Step 9 — Pivot confirmed (T1021.006)

```
[server] sliver (SACRED_EVENING-WEAR) >

[*] Session 196725b2 SACRED_EVENING-WEAR - 192.168.182.137:53848 (THEPUNISHER) - windows/amd64 - Tue, 20 Oct 2
026 12:50:57 CEST


[server] sliver (SACRED_EVENING-WEAR) > sessions

 ID         Transport   Remote Address           Hostname      Username      Operating System   Health
=========  ==========  ======================  ============  ============  =================  ========
 54cde061   http(s)     192.168.182.138:64324   SPIDERMAN     peterparker   windows/amd64      [ALIVE]
 196725b2   http(s)     192.168.182.137:53848   THEPUNISHER   frankcastle   windows/amd64      [ALIVE]

[server] sliver (SACRED_EVENING-WEAR) > use 196725b2

[*] Active session SACRED_EVENING-WEAR (196725b2-242b-43a9-9544-f7a569db1fc4)

[server] sliver (SACRED_EVENING-WEAR) >
```

A second C2 session on THEPUNISHER alongside the SPIDERMAN session.

Step 10 — LSASS dump via SharpKatz (T1003.001)

```
[server] sliver (SACRED_EVENING-WEAR) > sessions

 ID         Transport   Remote Address           Hostname      Username      Operating System   Health
=========  ==========  ======================  ============  ============  =================  ========
 54cde061   http(s)     192.168.182.138:64324   SPIDERMAN     peterparker   windows/amd64      [ALIVE]
 196725b2   http(s)     192.168.182.137:53848   THEPUNISHER   frankcastle   windows/amd64      [ALIVE]

[server] sliver (SACRED_EVENING-WEAR) > use 196725b2

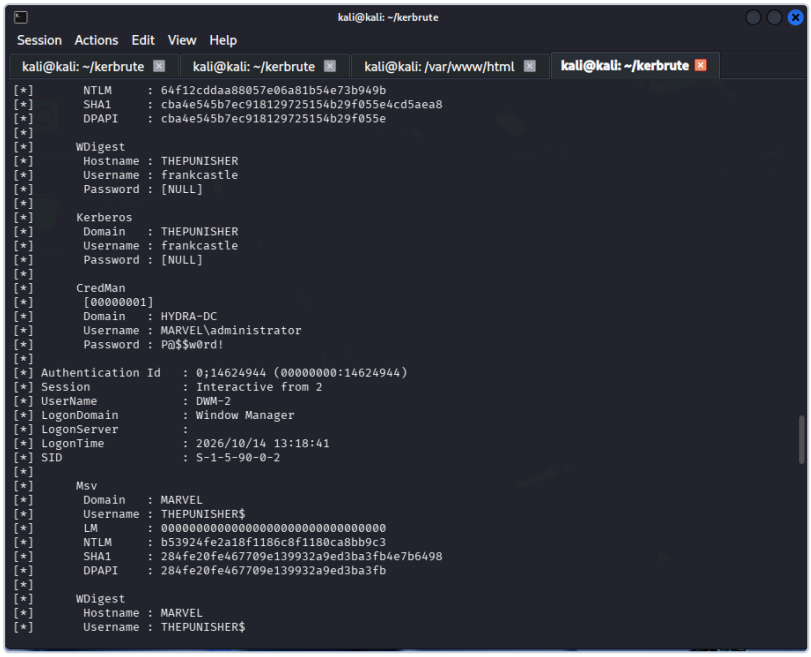
[*] Active session SACRED_EVENING-WEAR (196725b2-242b-43a9-9544-f7a569db1fc4)

[server] sliver (SACRED_EVENING-WEAR) > execute-assembly -t 120 /var/www/html/SharpKatz.exe --Command logonpas
swords

[*] Output:
[*]
[*]                    System Information
[*] ----------------------------------------------------------
[*] | Platform: Win32NT                                      |
[*] ----------------------------------------------------------
[*] | Major: 10           | Minor: 0           | Build: 19045 |
[*] ----------------------------------------------------------
[*] | Version: Microsoft Windows NT 10.0.19045.0             |
[*] ----------------------------------------------------------
[*]
[*] Authentication Id   : 0;32202442 (00000000:32202442)
[*] Session             : Network from 0
[*] UserName            : SQLService
[*] LogonDomain         : MARVEL
[*] LogonServer         :
[*] LogonTime           : 2026/10/20 12:42:51
[*] SID                 : S-1-5-21-1422081359-1692309262-2219121369-1104
[*]
[*]      Kerberos
[*]       Domain   : MARVEL.LOCAL
[*]       Username : SQLService
[*]       Password : [NULL]
[*]
```

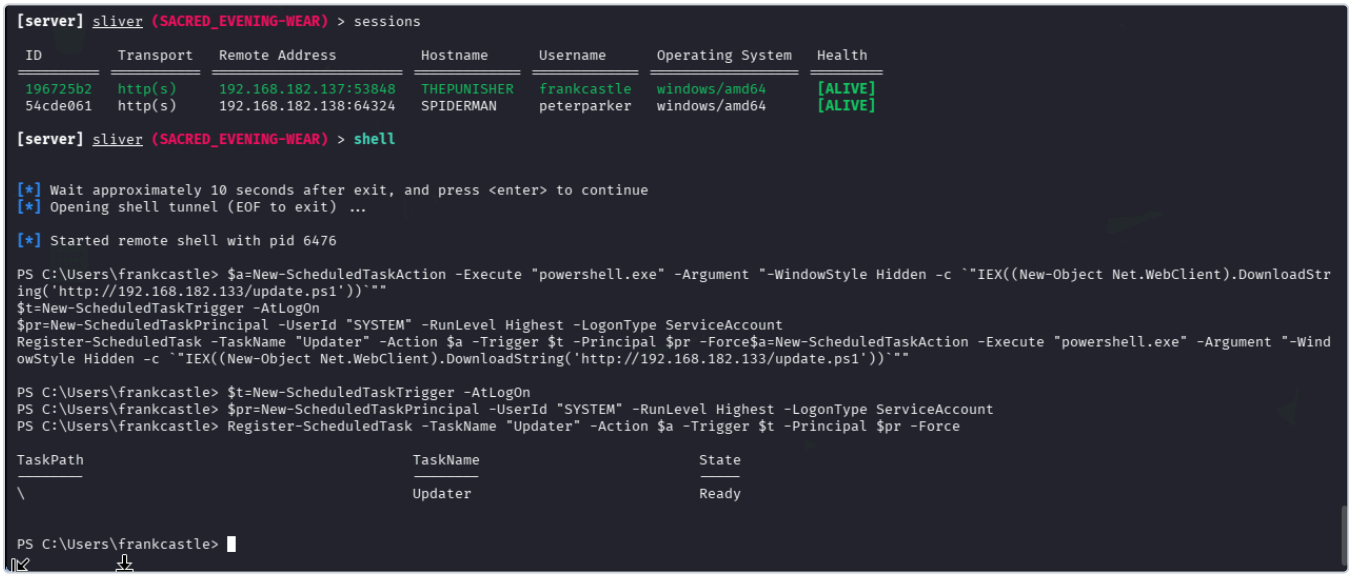
SharpKatz run in memory through Sliver on THEPUNISHER. This is the LSASS access caught by Sysmon EID 10 in detection 07, and missed by the EDR for the reasons in section 6.4.

Step 11 — Credentials recovered (T1003.001)



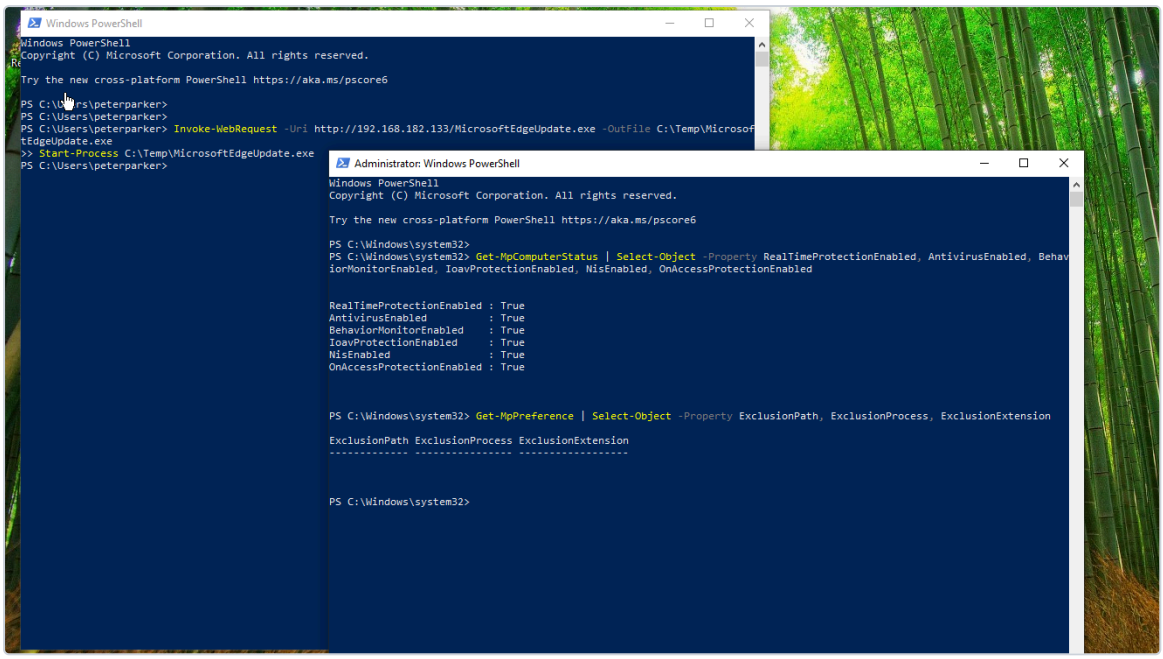
NTLM and SHA1 material, a cleartext `MARVEL\\administrator` credential, and the `THEPUNISHER$` machine account.

Step 12 — Persistence, scheduled task (T1053.005)



`Register-ScheduledTask -TaskName "Updater"` running `powershell.exe` as SYSTEM at logon. Its `4698` creation event is detection 08.

Step 13 — Defender enabled throughout (control state)



`Get-MpComputerStatus` shows real-time protection and all engines on, and `Get-MpPreference` shows no exclusion paths. A change from v2, which kept `C:\\Temp` excluded so delivery was reproducible on camera.

5. Detection coverage

Phase	Technique	Recorded by
Password spray	T1110.003	Windows `4771`, Zeek `kerberos.log`, Suricata `weak_encryption`
Kerberoasting	T1558.003	Windows `4769`, Suricata `krb5` and `weak_encryption`
Pass-the-Ticket	T1550.003	Windows `4768` TGT for `SQLService` (RC4), Zeek `kerberos.log`, Suricata `weak_encryption`
Lateral movement	T1021.006	Zeek `conn.log` port 5985, Suricata `REQUEST_AUTH_UNRECOGNIZED` (corroborating only)
LSASS access	T1003.001	Sysmon EID 10, THEPUNISHER only. Not by the EDR, see 6.4
Persistence	T1053.005	Windows `4698`, Wazuh `60228` at level 4
C2 beaconing	T1071.001	Zeek `ssl.log`, JA4 TLS fingerprinting

The column is what *recorded* each phase, not what alerted on it, and the two are not the same here. The Windows, Sysmon and Wazuh entries are rule coverage, shipped as Sigma, a Splunk app and `local_rules.xml`. The Zeek entries are rule coverage too, via three Zeek Sigma rules, though the sensor itself runs stock — default scripts, default logs, no `local.zeek` customisation, so there is nothing sensor-side to ship. The Suricata entries are not rule coverage at all: `krb5` is protocol logging and `weak_encryption` / `REQUEST_AUTH_UNRECOGNIZED` are built-in app-layer anomaly events. No signature was written or tuned for them, and no Suricata ruleset ships with this project. They are a second, independent source agreeing with the Windows and Zeek evidence, which is worth having and is not the same as a detection.

What weak_encryption actually fires on

That row appears three times above, and getting it right took a measurement that overturned the obvious reading. Every Suricata `WEAK_ENCRYPTION` between 11:00 and 13:00 on the run date — 25 events:

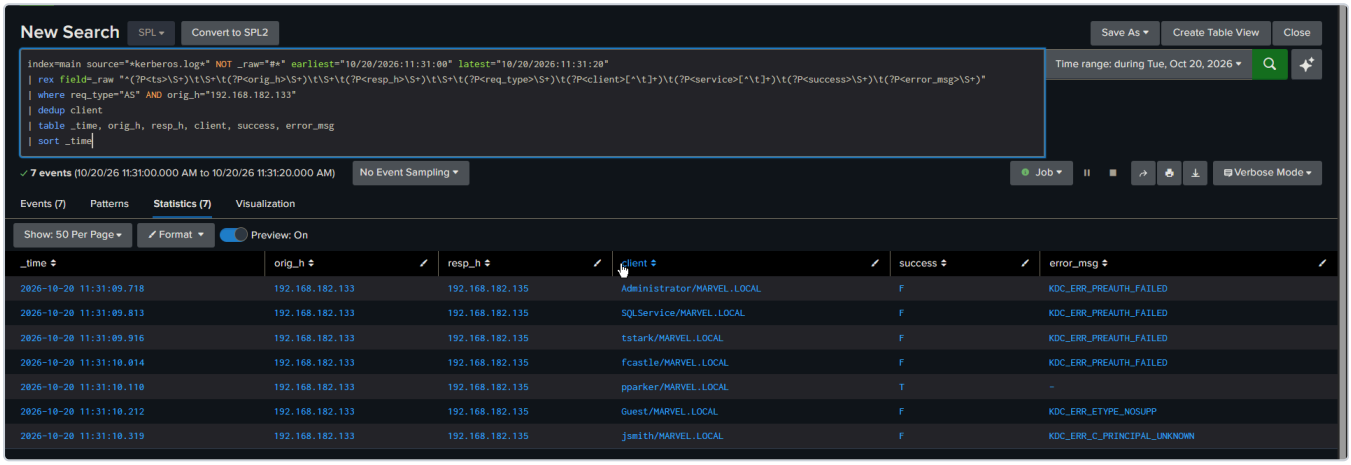
Time	Count	Direction	Chain step
11:19:17	1	`.138` → `.135`	—
11:31:09–10	11	`.135` → `.133`	password spray
11:32:48	5	`.133` → `.135`	Kerberoasting
12:20:54	1	`.138` → `.135`	—
12:31:18	3	`.135` ↔ `.137`	—
12:38:45	2	`.135` → `.138`	Pass-the-Ticket
12:42:51	2	`.138` → `.135`	WinRM pivot

It fires during the Kerberoasting, even though that ticket was AES-256. The reasonable guess — that an event named `weak_encryption` keys on the encryption type of the issued ticket, so an `0x12` roast cannot trip it — is wrong. Suricata raises it on a weak etype seen anywhere in the Kerberos exchange, including the client's *proposed* etype list. `GetUserSPNs` offers RC4 among its supported types; the KDC picks AES and issues `0x12`. Windows `4769` records only the outcome, Suricata sees the negotiation. That is finding 6.6 reached from the network side: RC4 is still on the table in this domain, and the host log makes the estate look AES-clean because it only ever shows what was issued.

It is also not high-fidelity. Five of the 25 fall on no attack step at all — ordinary domain traffic offering RC4. One in five is a false positive inside a known-bad window, which is why this sits in a *recorded by* column rather than being written up as a detection.

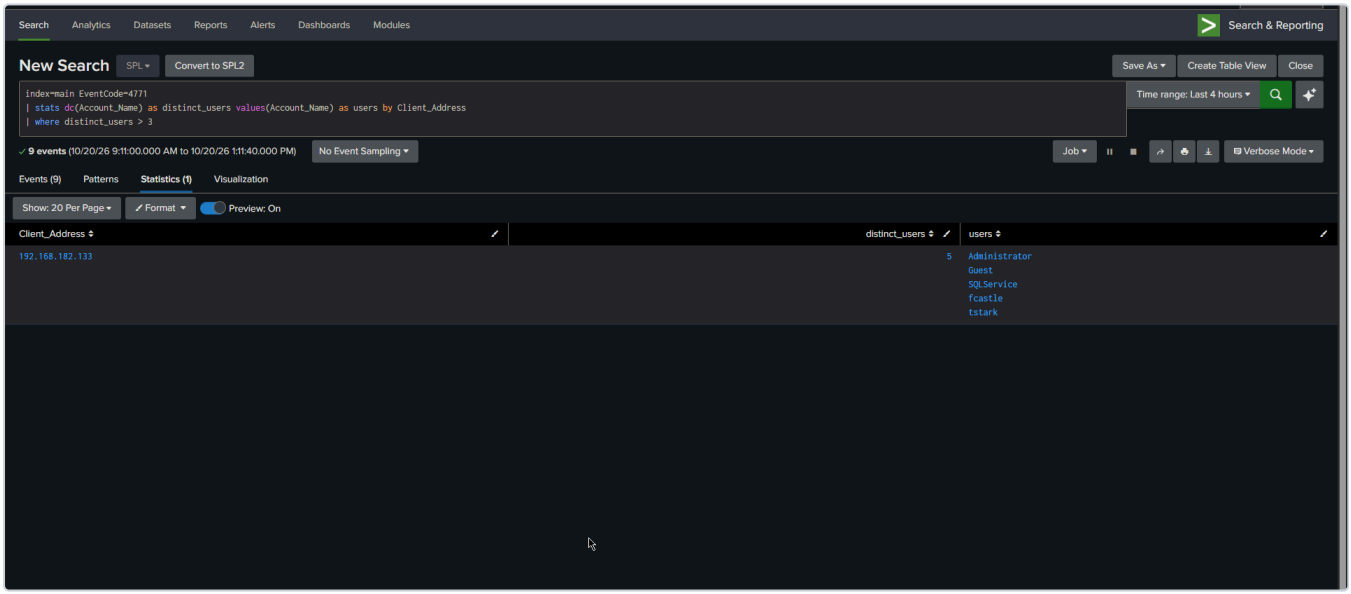
All times from the 20 October 2026 run. Zeek queries parse the raw TSV with `rex` to show log structure; in production the Zeek add-on extracts these fields, which is scoped into v4.

Detection 01 — Password spray, network view (T1110.003)



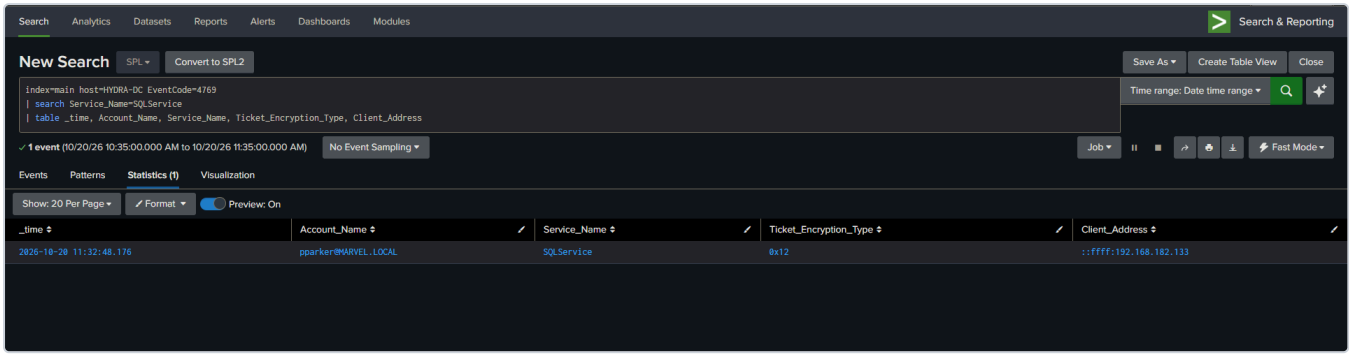
Zeek `kerberos.log`: seven accounts from `192.168.182.133` in 0.6 s. `pparker` succeeds. `jsmith` fails as unknown principal, which produces no Windows `4771` at all.

Detection 02 — Password spray, endpoint view (T1110.003)



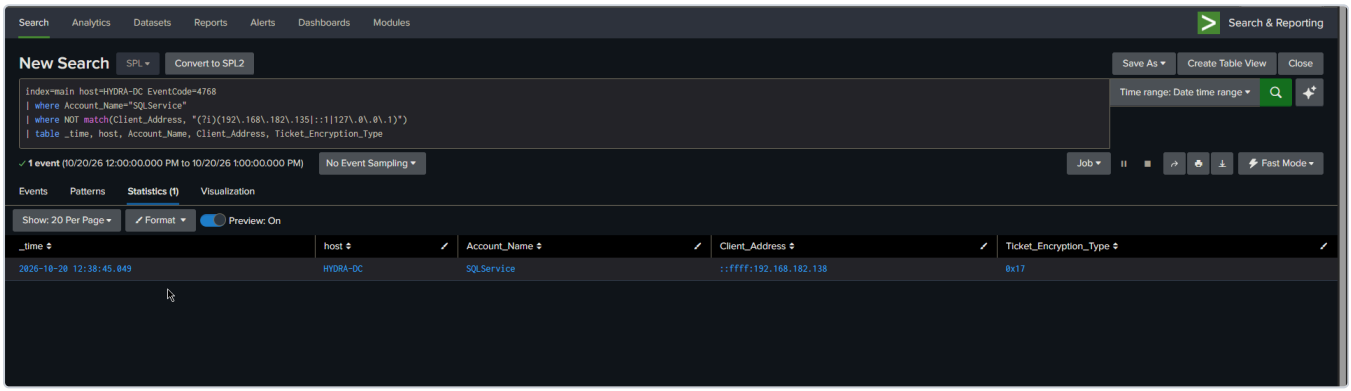
Windows `4771` correlation: single source, five accounts failing pre-authentication. Two fewer than the network layer saw.

Detection 03 — Kerberoasting, AES service ticket (T1558.003)



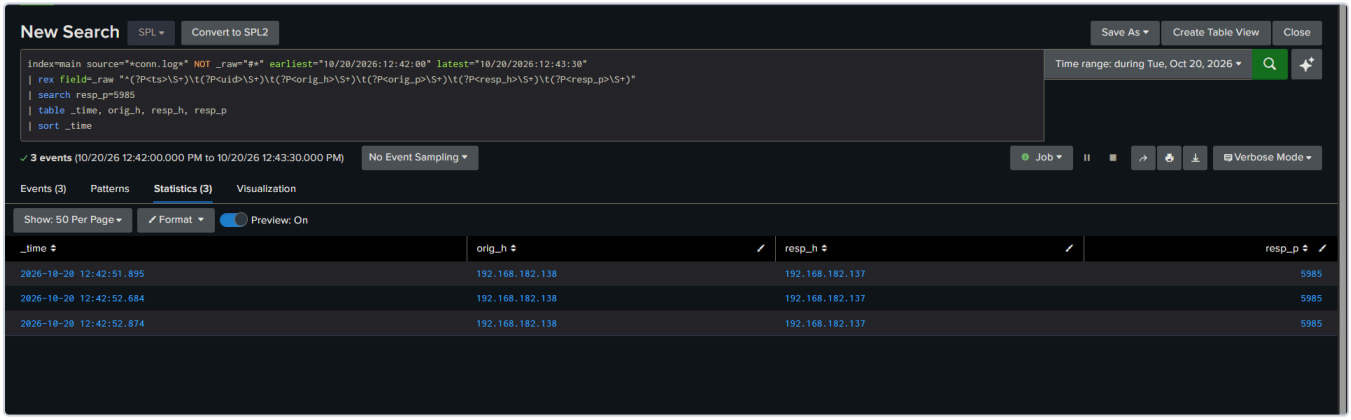
`4769` for `SQLService`, `Ticket_Encryption_Type=0x12` (AES-256), requested by `pparker`. The shipped RC4 rule keys on `0x17` and stayed silent.

Detection 04 — Pass-the-Ticket (T1550.003)



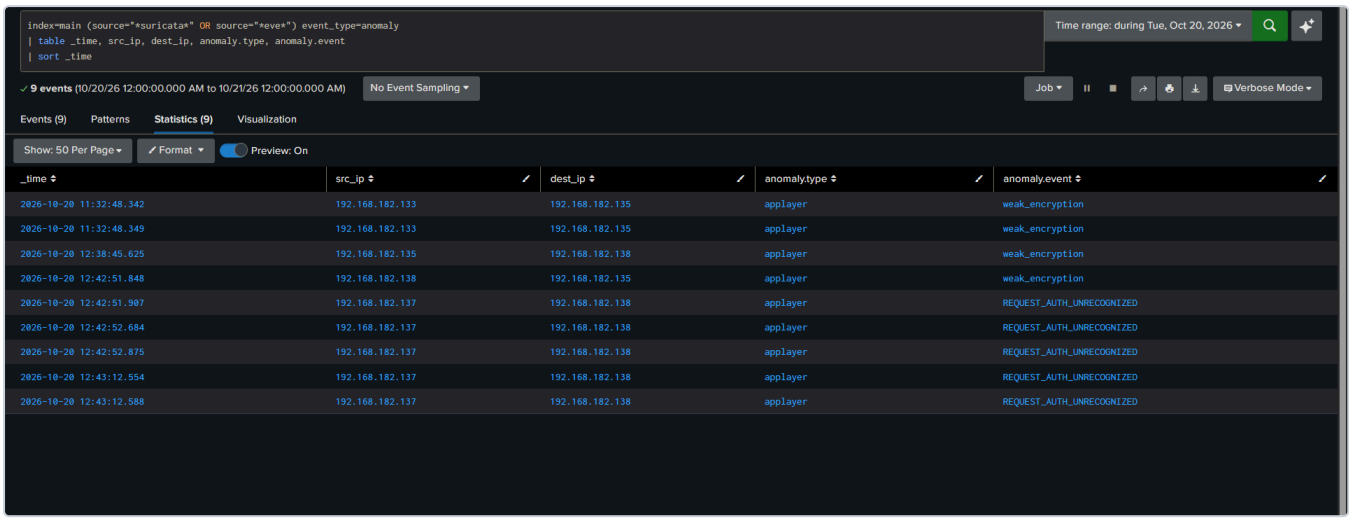
`4768` TGT requested *for* `SQLService` from SPIDERMAN at 12:38, RC4. `4768` rather than `4769` is the tell: a roast requests a service ticket, this requests a ticket-granting ticket in the account's own name, which means the credential is already held.

Detection 05 — Lateral movement over WinRM (T1021.006)



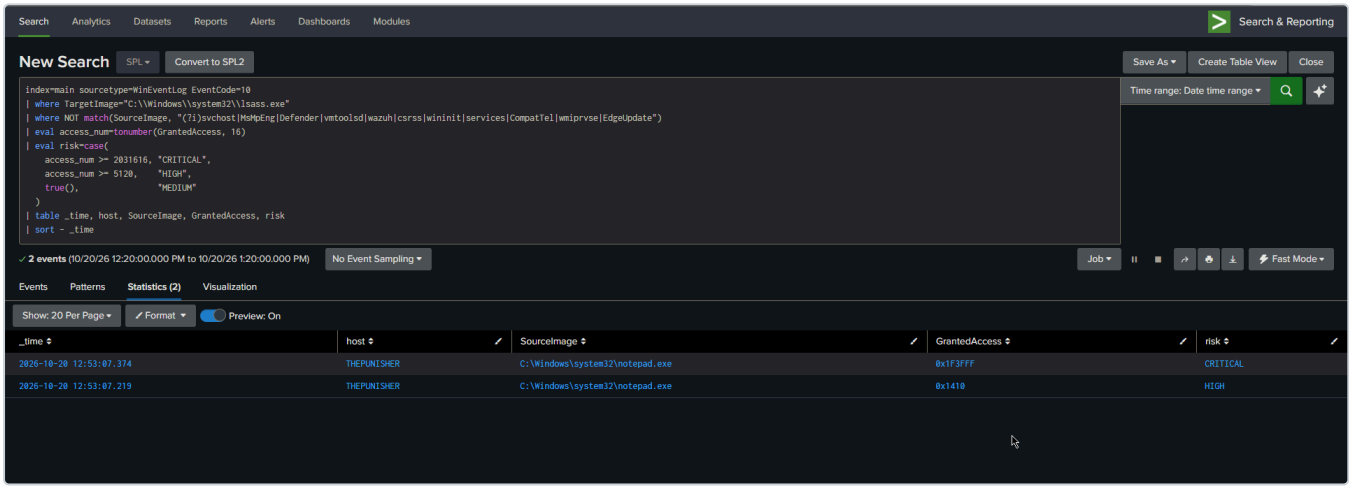
Zeek `conn.log`: three connections `192.168.182.138` to `192.168.182.137` on port 5985. Workstation to workstation, which has no legitimate reason in an estate with no jump host.

Detection 06 — Suricata application-layer anomalies



Suricata's built-in app-layer anomaly events, not written signatures: `weak_encryption` across the Kerberos exchanges -- spray, roast and Pass-the-Ticket alike, because it keys on the etypes offered rather than the ticket issued, and it is 5-in-25 false positives over the run window -- and `REQUEST_AUTH_UNRECOGNIZED` during the 12:42 to 12:43 WinRM pivot. Both corroborate the Windows and Zeek evidence rather than standing on their own — the second also fires on legitimate WinRM, since `Negotiate` is an `Authorization` header Suricata's HTTP parser does not recognise.

Detection 07 — LSASS access (T1003.001)



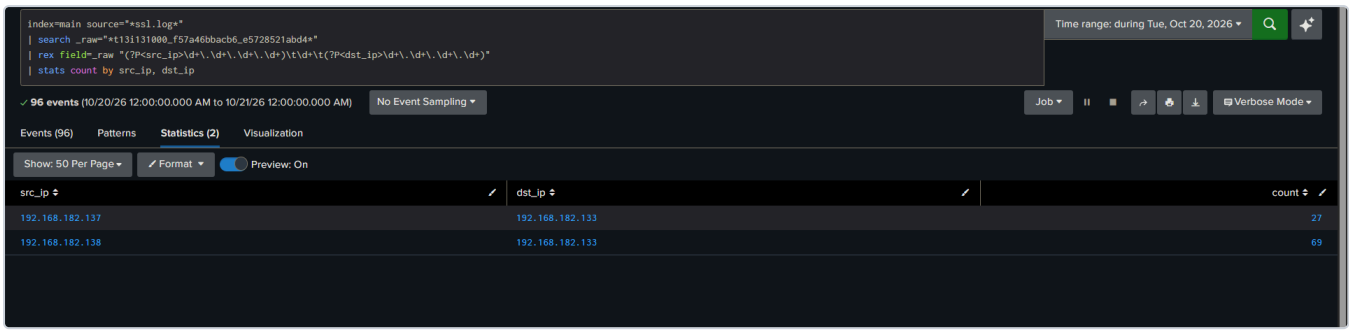
Sysmon EID 10 on THEPUNISHER: `notepad.exe` accessing `lsass.exe` with `GrantedAccess 0x1F3FFF`. The process name is a disguise; SharpKatz ran in memory inside a host process. The query excludes `MsMpEng.exe` so Defender's routine access is filtered out.

Detection 08 — Persistence, scheduled task (T1053.005)



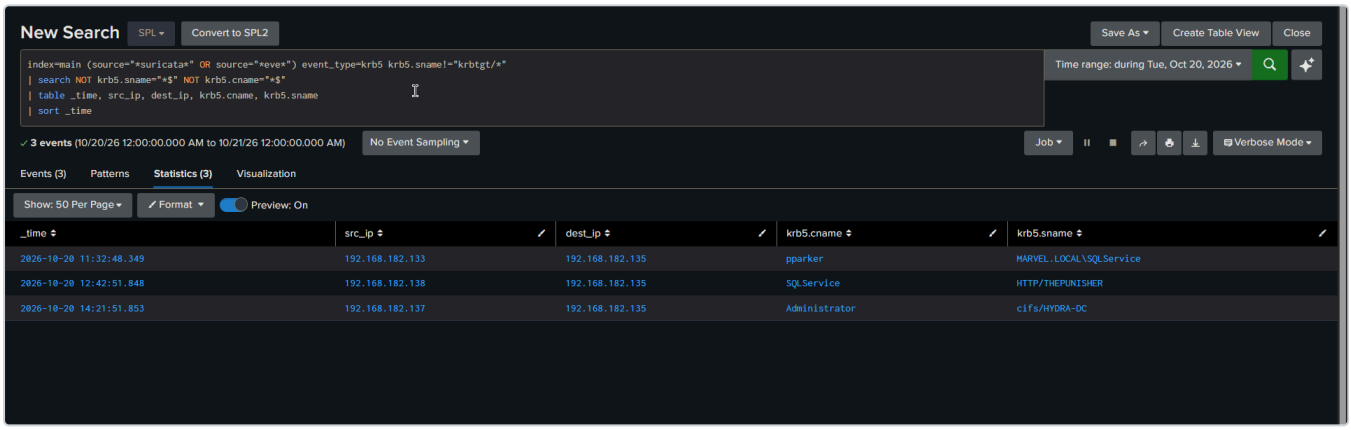
`4698` on THEPUNISHER: task `\\Updater`, `powershell.exe -WindowStyle Hidden -c IEX(...)`. The download cradle is in the event body, which the EDR rule does not read.

Detection 09 — C2 beaconing, JA4 fingerprint (T1071.001)



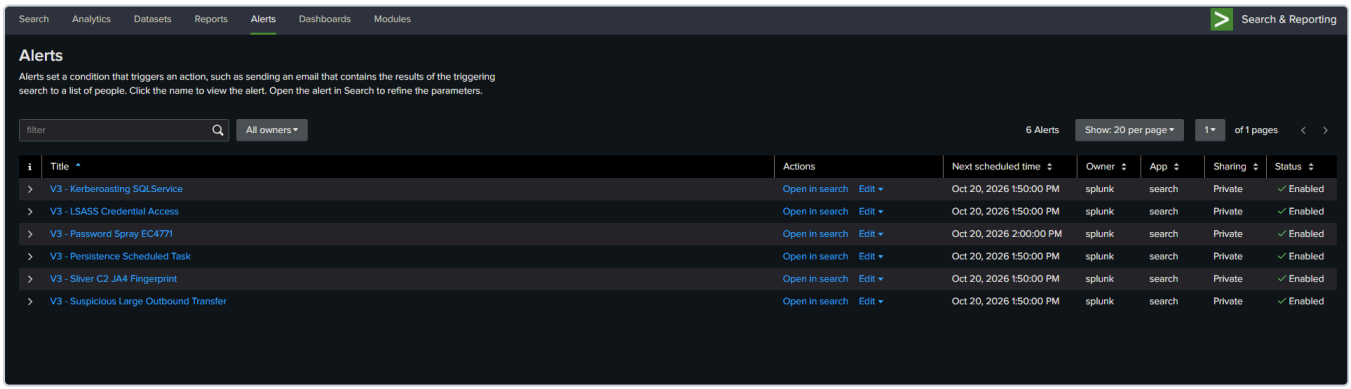
Zeek `ssl.log` filtered on the Sliver JA4 `t13i131000_f57a46bbacb6_e5728521abd4`: 69 beacons from SPIDERMAN and 27 from THEPUNISHER. One fingerprint links two compromises that share no filename or hash. Note the prerequisite: `ja4` is not a base Zeek field. It comes from the FoxIO JA4 package (`zkg install zeek/foxio/ja4`), and without it `ssl.log` has no such column and this detection cannot fire at all.

Detection 10 — Kerberoasting by signature, not by address (T1558.003)



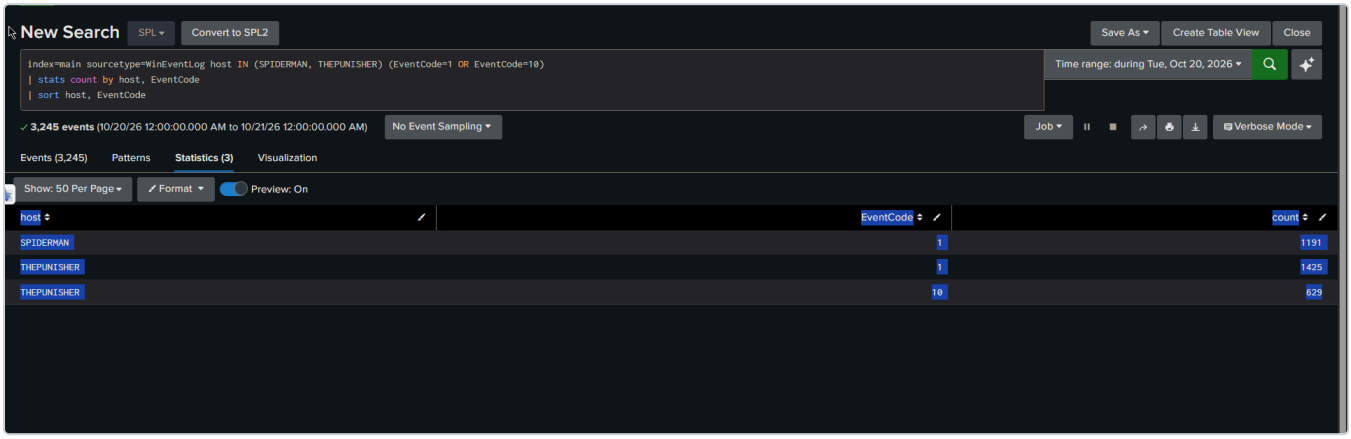
Suricata `krb5` filtered to service tickets whose SPN is neither `krbtgt` nor a machine account, so the filter keys on the pattern rather than the attacker's address. Three tickets surface the whole credential-abuse chain: the roast, the cracked account being reused against `HTTP/THEPUNISHER`, and `Administrator` against `cifs/HYDRA-DC`.

Detection 11 — The alerts running in the lab's own Splunk



Six v3 saved alerts, all enabled. These fire on schedule because v3 runs an Enterprise licence; on v2's Free licence the same searches run only on demand. This is the live instance, not the shipped app. When this screenshot was taken the two lists differed: the app had no Splunk search for the JA4 C2 beacon and none at all for the large-outbound-transfer alert, both written ad hoc here and never committed. Both were added afterwards, so the app now carries 15 searches with 10 scheduled and covers all six. The naming is settled on the app's `Purple - …`; the case files also record the `V3 - …` name each alert carried during the run, because what the analyst saw is part of the record.

Detection 12 — The SPIDERMAN blind spot, proven



SPIDERMAN forwarded 1,191 Sysmon process-create events and zero ProcessAccess events, while THEPUNISHER logged 629. `sysmon64 -c` on the host shows `ProcessAccess onmatch: include` with no rule lines under it, so the include list matches nothing. The same dump on that host would leave no trace. Found afterwards and it reverses the reading: that empty include is what the repo's own `config/sysmonconfig.xml` shipped, so SPIDERMAN was running the published baseline unmodified and THEPUNISHER was the exception, carrying an `lsass` line someone had added on the host and never folded back into the file. This was the config, not one host drifting, and any third endpoint would have been blind too. The LSASS rules were written, tuned against 935 sampled reads and documented before anyone checked that the shipped config emitted EID 10 at all. The config now carries the rule; re-deploying it and re-proving the event arrives is carried into v4.

Detection 13 — The Wazuh scheduled-task rule gap (T1053.005)

```
wazuh@wazuh-vm:~$ echo "=== Rule 60228 (scheduled task created) na THEPUNISHER — nivo + task ==="
=== Rule 60228 (scheduled task created) na THEPUNISHER — nivo + task ===
wazuh@wazuh-vm:~$ zcat /var/ossec/logs/alerts/2026/Oct/ossec-alerts-20.json.gz | python3 -I -c "
> import sys,json
> for line in sys.stdin:
>     try:
>         e=json.loads(line)
>         r=e.get('rule',{})
>         if r.get('id','')=='60228' and 'THEPUNISHER' in e.get('agent',{}).get('name',''):
>             t=e.get('data',{}).get('win',{}).get('eventdata',{}).get('taskName','')
>             print('level', r.get('level',''), '| rule', r.get('id',''), '|', t)
>     except: pass
> " | sort | uniq -c
      1 level 4 | rule 60228 | \\OneDrive Standalone Update Task-S-1-5-21-2864021089-2511867631-981569165-1001
      1 level 4 | rule 60228 | \\Updater
wazuh@wazuh-vm:~$
```

Rule `60228` fired at level 4 for the malicious `\\Updater` task and for a benign `\\OneDrive Standalone Update Task`. Same rule, same severity. The hidden PowerShell cradle inside `\\Updater` is in the event body and is not scored.

6. Findings

A note on timestamps. Two clocks appear in this report and only one is labelled. The rule-verification times (24 October, `11:36:58` onward, and Appendix A) are UTC, taken from Wazuh. The attack timeline (20 October, `11:31` through `12:43`) is as rendered in Splunk, which displays in the instance's own timezone, and the Windows hosts run UTC+2. The two sets are therefore not necessarily on the same clock and nothing here reconciles them; the attack narrative is internally consistent because every time in it comes from the same render. No finding depends on the offset — the intervals that carry meaning (98 seconds from spray to roast, four minutes from Pass-the-Ticket to the WinRM pivot) are all inside the attack timeline.

6.1 Zeek recorded a spray attempt Windows did not

Zeek `kerberos.log`, 11:31:09.718 to 11:31:10.319, source `192.168.182.133`:

Outcome	Accounts
Succeeded	`pparker`
Pre-authentication failed	`Administrator`, `SQLService`, `tstark`, `fcastle`
Unsupported encryption type	`Guest`
Unknown principal	`jsmith`

Windows logged `4771` for five of these. The `jsmith` attempt produced none, because Kerberos emits no `4771` for an account that does not exist. The Splunk alert reported five accounts; seven were tried.

The shipped rule narrows it further by filtering `Failure_Code="0x18"`, wrong password. Counting every `4771` failure code in this lab over 30 days:

Code	Meaning	Count
`0x18`	wrong password	67
`0xE`	encryption type not supported	7
`0x25`	clock skew	2

The seven `0xE` events are spray attempts that do produce a `4771`, and the rule drops them. So the endpoint layer undercounts twice: once for accounts that generate no event, and again for accounts that fail on something other than the password.

6.2 One JA4 fingerprint linked both endpoints

Both endpoints beaconed to the attacker over TLS with the same Sliver JA4 client fingerprint `t13i131000_f57a46bbacb6_e5728521abd4`: 69 sessions from SPIDERMAN, 27 from THEPUNISHER. The two implants were delivered at different times by different routes and share no filename or hash. The fingerprint is derived from how the TLS client is built, so renaming the binary does not change it.

It remains an indicator. It is tied to this Sliver build's TLS stack, and a different C2 or a changed client profile produces a different fingerprint. More durable than a hash, less durable than behaviour.

6.3 Wazuh scores task creation, not task content

The `\Updater` persistence task fired rule `60228` at level 4. A benign `\OneDrive Standalone Update Task` on the same host fired the same rule at the same level. The hidden PowerShell download cradle is in the `taskContent` field of the event, and nothing reads it.

Level 4 is below where an analyst working a severity-sorted queue looks. The event was received, parsed and alerted on, and the output was still useless.

6.4 Wazuh rule 92900 fired 32 times with 0 true positives

`92900` is Wazuh's built-in LSASS-access rule, level 12. Across all archived alert files:

Source	Count	Masks
`MsMpEng.exe` (Defender)	29	`0x101000`, `0x1010`
`svchost.exe`	2	`0x101000`
`MRT.exe`	1	`0x101000`

The SharpKatz read of LSASS is not among them. Reading the rule explains why:

```
<rule id="92900" level="12">
  <if_group>sysmon_event_10</if_group>
  <field name="win.eventdata.targetImage">(?i)lsass\.exe</field>
  <field name="win.eventdata.grantedAccess">(?i)(0x1010|0x40)</field>
```

`0x1010` is what Defender uses. `0x1fffff` and `0x1438` are not matched, and SharpKatz read LSASS at `0x1F3FFF`. The severity is inverted: wrong every time on the benign mask, silent on the aggressive one. Confirmed by trigger, a PowerShell `OpenProcess(lsass, PROCESS_ALL_ACCESS)` produced a Sysmon event and no Wazuh alert.

6.5 The EDR receives Sysmon from one of three agents

Agent	Alerts	From Sysmon	EID 10
HYDRA-DC	51,725	0	0
SPIDERMAN	22,782	0	0
THEPUNISHER	18,591	376	32

Only THEPUNISHER carries a `Microsoft-Windows-Sysmon/Operational` block in its `ossec.conf`. The DC case is the sharpest: Sysmon is installed and running there and its own log holds 43,683 events that the agent never reads. The agent reports healthy and busy while the EDR sees none of it.

The `4698` task rules are unaffected, because the Security channel is read everywhere.

6.6 The canonical Kerberoasting detection never saw its input

The RC4 rule is the one every guide teaches. Counted over 30 days of this lab's `4769` traffic:

Encryption type	Tickets
`0x12` (AES-256)	1,649
`0x17` (RC4)	0

Not one RC4 service ticket. The rule is scheduled, it appears in the coverage table, and it has never had anything to match. The v3 roast was issued as `0x12` and walked past it.

The reason is not that the domain refuses RC4. `SQLService` still holds an RC4 key and the KDC will hand one out: the Pass-the-Ticket step of this same run obtained a `4768` TGT for that account at `0x17` (detection 04). What happened is narrower. `GetUserSPNs` asked for no particular encryption type, the

account has AES keys, and the KDC issues the strongest etype both sides share, so the roast came back AES-256 on its own.

That changes what the rule is worth rather than retiring it. It still covers the downgrade variant — an attacker requesting RC4 explicitly (`Rubeus kerberoast /rc4opsec`, or an etype-restricted `TGS-REQ`) gets it on this domain. What it cannot see is the roast that asks for nothing special, which is what a current toolchain produces by default and what this lab ran. So two rules are needed, not one: the repo ships the etype-agnostic companion keyed on the SPN owner, which is noisier and needs a per-environment allowlist. Removing the account's RC4 key would close the downgrade path as well, and the `0x17` TGT above says that has not been done here — AES is being *preferred*, not *enforced*, which is a different posture than the ticket counts suggest on their own.

This sits alongside 6.4 as the other way a rule reports coverage without delivering it: there the rule could not load, here it loads and nothing in the run produces its input.

6.7 A rule tuned against the baseline rather than against the attack

A first draft of the LSASS replacement matched `0x1010` and `0x1410` alongside the full-access masks. Measured against 935 `ProcessAccess` reads of `lsass.exe` sampled from THEPUNISHER's own Sysmon log, it would have alerted 90 times:

Count	Mask	Source
79	`0x1410`	`WmiPrvSE.exe`
4	`0x1410`	`vmtoolsd.exe`
3	`0x1410`	`wazuh-agent.exe`
2	`0x1fffff`	`wazuh-agent.exe`, the EDR agent itself
1	`0x1410`	`taskmgr.exe`
1	`0x1f3fff`	`powershell.exe`, the only one worth triage

89 of 90 are ordinary platform behaviour. Restricting to full and MiniDumpWriteDump masks and excluding the sources measured above brings the same sample to one alert. A rule that is wrong 98.9 per cent of the time is not a detection, and the only way to know that in advance is to count.

6.8 The same mistake again, in the Zeek rule, found in a screenshot

The network Kerberoasting rule excluded machine accounts with `client|endswith: '$'`. Zeek writes the client as `principal/REALM`, so a machine account reads `THEPUNISHER$/MARVEL.LOCAL` and does not end in `$`. The exclusion matched nothing, and the rule alerted on every machine account's service ticket — in a real estate, most of the Kerberos traffic there is.

The evidence was already in this report. Detection 01 shows the `client` column as `Administrator/MARVEL.LOCAL`, `pparker/MARVEL.LOCAL`, `jsmith/MARVEL.LOCAL`. The field format had been on screen since the spray was written up. The rule was written against the shape of the Windows field instead, where `Account_Name` is the bare account name and a suffix test is correct. Same expression, different wire format, silently inert.

That makes three rules in this lab that reported coverage and could not deliver it: one bound to a rule group that did not exist, one whose input the environment never produced, and this one. Two were caught by running the rule. This one was caught by re-reading a screenshot. None of the three were caught by reading the rule.

7. Detection engineering

What ships

Artefact	Contents
Sigma	13 vendor-agnostic rules, 10 host and 3 network
Splunk app	15 saved searches, 10 scheduled: 11 on raw Windows sourcetypes, 2 on raw Zeek logs and 2 on CIM data models
Wazuh	5 custom rules: LSASS detection, suppression of the built-in false positive, and task-content scoring
Sysmon config	including the `NetworkConnect` gap fix from v2

The two failures worth documenting

A rule that could not load

The original LSASS rule was deployed against `<if_group>sysmon_event10</if_group>`. The real group is `sysmon_event_10`, one underscore away. A `grep` over the ruleset returns nothing for the misspelling, and no alert in any archive has ever carried it. The rule alerted zero times, ever.

Its silence was first read as "no attack has been run since it was added", which was also true, so two causes produced the same zero and the harmless one hid the real one. Wazuh had been reporting the fault on every start:

```
wazuh-analysisd: WARNING: (7610): Group 'sysmon_event10' was not found.
Invalid 'if_group'. Rule '100200' will be ignored.
```

A replacement that also could not fire

The first replacement bound to `<if_sid>92900</if_sid>`, reasoning that a concrete parent SID is more stable than a group name. That could not fire either: `92900` filters `grantedAccess` to `0x1010` or `0x40`, so a child requiring a full-access mask is mutually exclusive with its own parent. Caught by running it rather than by reading it.

Proof that the fix works

A controlled before and after fell out of the test run. The same trigger, from the same host, with the same mask, six minutes apart, with only the rule's parent changed between them:

Time (UTC)	Parent	Alert
11:36:58	`<if_sid>92900</if_sid>`	none
11:42:38	`<if_group>sysmon_event_10</if_group>`	level 12

Sysmon recorded both events. The full verification log is Appendix A.

Where the Splunk pack does and does not work

Component	State in this lab
`Endpoint.Processes` data model	populated; the LOLBin detection works

`Network_Traffic.All_Traffic` data model	**empty**; the C2 beacon detection cannot return a row
Data model acceleration	**off** on both, so `tstats` here is a raw scan
The other eleven searches	work, because they filter on `source=` rather than `sourcetype=`

The cause is the forwarder. Every Windows channel arrives as a bare `WinEventLog` sourcetype with the channel name only in `source`:

```
sourcetype = WinEventLog
source     = WinEventLog:Microsoft-Windows-Sysmon/Operational
```

The Sysmon add-on keys its CIM field aliases and eventtypes on the per-channel sourcetype. Under a bare `WinEventLog` none of it applies, so 444 EID 3 events in seven days reach the index with no `app`, `src`, `dest` or `dest_port`, and nothing tags them into the model.

That the other eleven rules survive this was not a deliberate choice at the time. It is the accident of having written them against `source=`. The fix is a forwarder change rather than a rule change, and it is scoped into v4.

> The affected search ships unscheduled with the reason written into its description. An empty data model and a broken rule look identical from the alert list, which is precisely why it has to be stated rather than quietly fixed.

8. Alert triage

Detections are half the job. The other half is what an analyst does when one fires, and that is the part a SOC role is measured on. Seven case files cover the chain phase by phase in one format: the alert, the questions asked, the queries run, the verdict, and the tuning and hardening that follows. They chain in the order the attack ran.

Case	Phase	Verdict
CASE-01	Password spray	True positive. Seven accounts in 0.6 s from a non-domain host, one succeeded, used 98 seconds later.
CASE-02	Kerberoasting	True positive. Domain Admin credential cracked offline; the canonical RC4 rule never fired.
CASE-03	Pass-the-Ticket	True positive. `4768` rather than `4769` separates the theft from the use.
CASE-04	Lateral movement	True positive. The firewall mesh chose the route; WinRM was the one open lane.
CASE-05	LSASS access	True positive on one host, undetectable on the other. Four layers of absent coverage.
CASE-06	Persistence	True positive delivered at level 4, below where anyone looks.
CASE-07	C2 beaconing	True positive. Two endpoints linked by one fingerprint across the whole run.

Three of them are about detections that did not work, which is why they are written out rather than summarised. CASE-02 covers a rule the roast walked past because the KDC chose AES, CASE-05 covers four separate reasons an EDR stayed silent on a credential dump, and CASE-06 covers an alert that arrived correctly and still told the analyst nothing.

Full text in `v3/investigations/` in the repository.

9. Known gaps carried into v4

The hardening baseline is partial and inconsistent

That is the starting point rather than an oversight. v4 rebuilds it to a CIS-aligned standard: consistent SMB signing, LSA Protection, WDigest and LLMNR off, and a GPO firewall baseline with a domain-isolation gate, with WinRM east-west left open as the monitored pivot. The chain is then re-run for a before and after coverage matrix.

Account lockout is enforced and the spray never reached it

One attempt per account per window is how spraying evades lockout. The weaker control here is the seven-character minimum password length, which v4 raises.

The service account is a human-managed Domain Admin with an SPN

The roast cracks straight to Domain Admin rather than to a low-privilege service account, and the password was also stored in the account's AD `Description` field, which is a second exposure needing no cracking. v4 moves it to a gMSA and clears the description.

The Wazuh gaps are fixed at the rule layer, not the telemetry layer

The rules are deployed and verified, so `\Updater` now scores level 12 and a full-access LSASS handle alerts at all. The ingestion gap is not fixed: the LSASS rule has telemetry on one of three hosts. A correct rule over missing data is still no coverage.

The forwarder sends a bare sourcetype

Measured, not planned. Per-channel sourcetypes with `renderXml = 1`, then acceleration enabled, then both `tstats` detections re-run with a before and after. "The data model is empty" and "the rule is wrong" look identical from the alert list, so the fix has to be demonstrated rather than assumed.

Zeek is parsed positionally

v3 queries parse the raw TSV with `rex`, which shows log structure but breaks on an empty or shifted column. v4 switches Zeek to JSON and ingests through the Splunk add-on, so fields are extracted by the TA.

The shipped Sysmon config could not feed the LSASS detection

The baseline ships `ProcessAccess` as an `onmatch="include"` with no rule lines, and an include with no rules logs nothing. SPIDERMAN's blind spot was that config running as published, not a drifted host; THEPUNISHER only produced EID 10 because the `lsass` line had been added there by hand. A Sigma rule, a Splunk search and three Wazuh rules were built and tuned against telemetry the published config did not emit. The rule is now in `config/sysmonconfig.xml`, but re-deploying it on both endpoints and re-proving that the event arrives is v4 work, so the before/after this report insists on elsewhere does not exist for this fix yet.

Two of the lab's six alerts never became rule content

The JA4 C2 detection ships as a Sigma rule but has no Splunk search, and the large-outbound-transfer alert was written ad hoc in the lab and was never committed at all. The lab also names its alerts `V3 - …` while the shipped app names them `Purple - …`, so an alert name quoted in a case file matches nothing

a reader can deploy. v4 ships both searches and settles on one naming scheme across the app, the Sigma pack and the case files — the case files are only useful if someone else can follow them.

Appendix A — Rule verification log

24 October 2026. Every rule deployed and fired by a targeted trigger rather than assumed to work. Times UTC; the Windows hosts run UTC+2.

Rule	Trigger	Result
—	`wazuh-analysisd -t`	clean; the previous version raised `(7610)` and discarded its rule
`100200`	PowerShell `OpenProcess(lsass, PROCESS_ALL_ACCESS)`	level 12 at 11:42:39
`100201`	20 platform `ProcessAccess` reads forced through the chain	0 alerts
`100205`	`MsMpEng.exe` reading LSASS at `0x1010`	0 alerts, where it was level 12 thirty-two times out of thirty-two
`100210`	task with `IEX` and `DownloadString`	level 12 at 11:48:57
`100211`	task with `-nop -w hidden -enc`	level 10 at 11:49:04
negative	task running `defrag.exe C: /O`	stays `60228`, level 4
negative	task running `powershell.exe -ExecutionPolicy Bypass -File ...`	stays `60228`, level 4

The second negative test is the one that counts. A scheduled task that legitimately runs PowerShell with `-ExecutionPolicy Bypass` is the realistic false positive for these rules, and neither fired on it. Writing a rule that alerts is easy; writing one that stays quiet when it should is the part worth testing.

`100201` has no direct trigger, because no platform process in this lab opens LSASS with a full-access mask. It was tested by temporarily widening `100200` to include `0x1000`, the mask `svchost.exe` and `MsMpEng.exe` use constantly, so ordinary platform reads had to pass through the suppression chain. Twenty such events produced zero alerts, and the mask was then reverted.

Appendix B — Measurement data

LSASS access masks present in the index

Sysmon EID 10 against `lsass.exe`, counted over seven days:

Mask	Count	In the shipped rule
`0x1000`	1,498	no
`0x1410`	177	no, this is the 79-event WmiPrvSE noise plus others
`0x1400`	107	no
`0x101000`	40	no
`0x1010`	25	no
`0x1418`	4	no
`0x1FFFFF`	8	**yes**
`0x1F3FFF`	2	**yes**, this is the attack

The shipped mask list is `0x1438`, `0x143a`, `0x1f0fff`, `0x1f1fff`, `0x1f2fff`, `0x1f3fff` and `0x1fffff`, identical across the Sigma rule, the Splunk search and the Wazuh rule.

The nine excluded readers are listed as full paths rather than as `\svchost.exe`. A filename-suffix exclusion also suppresses `C:\Temp\svchost.exe`, which is how a masquerading implant would hide from the rule meant to catch it. The Sigma rule and the Splunk search both matched on suffix until this was checked, and the Sigma list carried `wininit.exe` and `csrss.exe` that the other two did not; all three now use the same nine anchored paths. The Wazuh pattern is tested against the real paths and against a fake `svchost.exe` in a user-writable directory, which still alerts. What none of them catch is code injected into the genuinely signed `svchost.exe` or `MsMpEng.exe`, which inherits a legitimate `SourceImage`; separating that needs `CallTrace` and this lab has no sample to test it against.

Service ticket encryption types, `4769` only, 30 days

Type	Count
`0x12` (AES-256)	1,649
`0x17` (RC4)	0

Scoped to `4769` deliberately. It is not a statement about the domain's Kerberos posture as a whole: the `4768` TGT obtained during Pass-the-Ticket came back `0x17`, so RC4 is available on request. See 6.6.

Kerberos pre-authentication failure codes, 30 days

Code	Meaning	Count
`0x18`	wrong password	67
`0xE`	encryption type not supported	7
`0x25`	clock skew	2

Sysmon event IDs present on SPIDERMAN

Grouping 5,000 recent events by ID returns eleven distinct values: 1, 2, 3, 4, 5, 8, 11, 12, 13, 22 and 255. EID 10 does not appear.

Splunk environment

Item	Value
Licence group	Enterprise, so scheduled searches and alerting are available
Sourcetypes in the index	`WinEventLog` 330,188 · `suricata` 139,759 · `zeek` 131,888 · `wazuh` 33,042
`Endpoint.Processes`	populated
`Network_Traffic.All_Traffic`	0 events
Acceleration	not enabled on either model

> Every figure in this appendix came from a query run against the live index or from reading the running configuration on the host, not from the attack run's own output. Where a number could not be confirmed that way, it is not here.